

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Объектно-ориентированное программирование»

«К защите допустить»
Руководитель курсового проекта
ассистент кафедры информатики
_____ А.Г. Бокун
____.____.2025

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
курсового проекта
на тему:
**«ВЕБ-ПРИЛОЖЕНИЕ ДЛЯ ОЦЕНКИ И РЕЦЕНЗИРОВАНИЯ
ФИЛЬМОВ»**

БГУИР КП 6-05-0612-02 06 ПЗ

Выполнил студент группы 353503
БОЧКОВ Александр Александрович

(подпись студента)

Курсовой проект представлен на
проверку _____.____.2025

(подпись студента)

Минск 2025

СОДЕРЖАНИЕ

Введение	5
1 Постановка задачи	6
1.1 Структура и архитектура платформы	6
1.2 История, версии и достоинства	6
1.3 Обоснование выбора платформы	7
1.4 Анализ существующих аналогов.....	7
2 Теоретическое обоснование разработки	10
2.1 Обоснование необходимости разработки.....	10
2.2 Технологии программирования, используемые для решения поставленных задач	11
2.3 Связь архитектуры ООП с разрабатываемым проектом.....	14
3 Функциональные возможности	17
3.1 Описание структуры базы данных приложения	17
3.2 Описание функций программного обеспечения приложения с учетом выбранной темы курсового проекта	19
3.3 Описание функциональной схемы программы.....	26
4 Описание использования приложения.....	28
Заключение	35
Список используемых источников.....	36

ВВЕДЕНИЕ

Одним из самых популярных развлечений на сегодняшний день является кинематограф. Ежегодно в мире выпускаются тысячи новых картин. Это создаёт потребность в платформах, которые будут предоставлять актуальную информацию о кинопроизведениях. Помимо этого, такие платформы способствуют формированию сообщества кинолюбителей, позволяя им делиться своим мнением.

Целью данной курсовой работы является проектирование и разработка веб-приложения на платформе ASP.Net Core MVC с возможностью публикации отзывов на кинофильмы, получения информации о них и категоризации произведений из в личных списках.

Задачи, которые необходимо выполнить в данной работе:

- 1 Изучение существующих аналогов.
- 2 Проектирование архитектуры приложения.
- 3 Разработка базы данных для приложения.
- 4 Создание интуитивного пользовательского интерфейса.

В приложении должен быть реализован функционал:

- 1 Регистрация новых пользователей с ограничением прав по доступу.
- 2 Возможность добавления информации о новых фильмах модератором.
- 3 Добавление пользователем фильмов в свои списки.
- 4 Оценка пользователем фильмов

Веб-приложение сможет охватить широкую аудиторию пользователей: как тех, кто просто ищет информацию о новинках в сфере кинематографа, так и тех, кто хочет систематизировать и обсудить кинофильмы.

Веб-приложение будет разработано на языке C# и платформе ASP.Net Core MVC. Разметка и стилизация веб-страниц будет выполнена при помощи HTML, CSS.

Пояснительная записка оформлена в соответствии с СТП 01-2024 [1].

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Структура и архитектура платформы

ASP.NET Core MVC - фреймворк с открытым исходным кодом для создания веб-приложений, разработанный Microsoft. Он предназначен для кроссплатформенной работы, что означает, что он может работать на Windows, Linux и macOS. Архитектура ASP.NET Core MVC предполагает, что элементы проекта будут разделены на три части: модели, представляющие таблицы в базах данных, представления, необходимые для генерации html-страниц и контроллеры, обрабатывающие http-запросы.

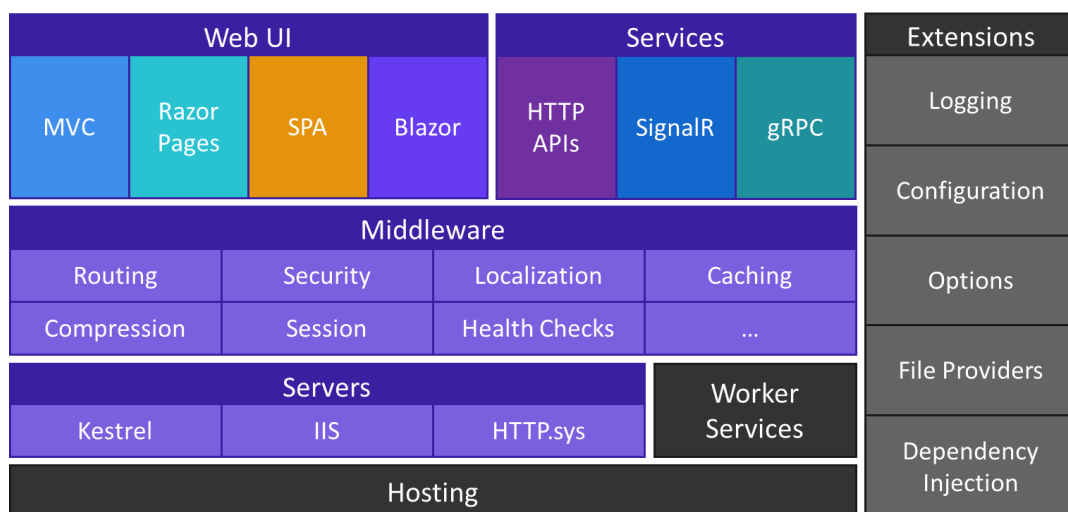


Рисунок 1.1 – Общие сведения о ASP.NET Core

1.2 История, версии и достоинства

ASP.NET Core — это кроссплатформенный, высокопроизводительный, открытый фреймворк для создания современных, облачных и подключенных к Интернету приложений. Он был представлен в июне 2016 года как переработка ASP.NET, объединяющая ранее отдельные ASP.NET MVC и ASP.NET Web API в единую модель программирования. Отличия ASP.NET Core от ASP.NET:

1 Кроссплатформенность: приложения работают на Windows, Linux и macOS.

2 Поддерживает не только MVC, но и более модульные архитектуры.

3 Улучшенная поддержка облака.

Преимущества ASP.NET Core MVC включают:

1 Открытый исходный код.

2 Генератор шаблонов HTML Razor Pages.

1.3 Обоснование выбора платформы

Платформа ASP.NET Core MVC была выбрана для разработки веб-приложения по следующим причинам:

1 Экосистема .NET: при разработке приложения возможно использование множества готовых библиотек и решений.

2 Знакомство с технологией .NET и языком программирования C#.

3 Производительность: платформа ASP.NET Core MVC предназначена для создания высоконагруженных веб-приложений с возможностью масштабирования.

4 Простота архитектуры: ASP.NET Core MVC реализует простую, но проверенную и эффективную архитектуру модель-представление-контроллер. Несмотря на то, что данный шаблон может считаться несколько устаревшим, он достаточно прост для того, чтобы изучить и на практике реализовать базовые паттерны проектирования.

1.4 Анализ существующих аналогов

Среди них можно выделить IMDb, Кинопоиск и Shikimori. Данные сервисы, помимо обширной базы данных, предоставляют площадку для публикации статей в сфере кинематографа, освещают различные события, реализуют сервис по продаже билетов. Рассмотрим данные сервисы подробнее.

Internet Movie Database (IMDb) — веб-сайт с крупнейшей в мире свободно редактируемой базой данных о кинематографе.

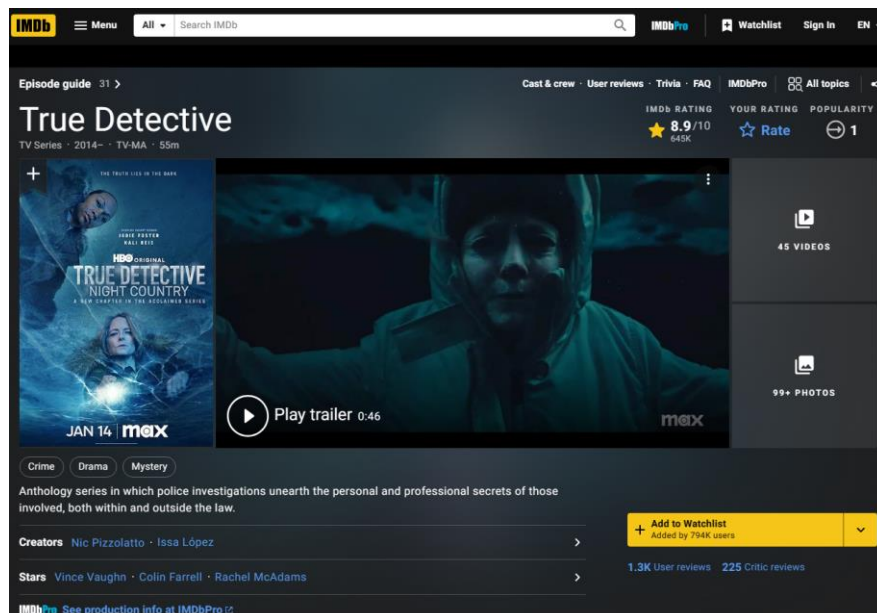


Рисунок 1.2 — пользовательский интерфейс IMDb

Преимущества IMDb:

- 1 Свободное программное обеспечение.
- 2 Поддержка множество языков (русский не поддерживается).
- 3 Возможность оценки отдельных эпизодов сериалов.
- 4 Наличие краткого описания сюжета, скриншотов, интересных фактов на странице фильма.

5 Наличие справочной информации о фильме: пояснение сюжета, список использованных композиций.

Недостатки IMDb:

- 1 Перегруженный пользовательский интерфейс.
- 2 Необходимость регистрации на веб-сайте для получения доступа к полной информации о произведении.

Кинопоиск — крупнейший русскоязычный интернет-сервис с условно свободно редактируемой базой данных, интернет-издание о кинематографе, онлайн-кинотеатр.

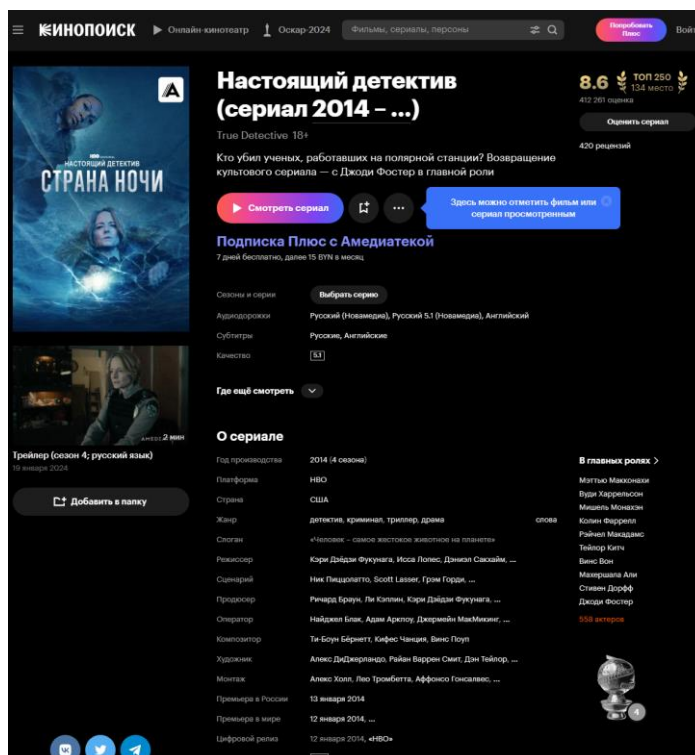


Рисунок 1.3 — пользовательский интерфейс сервиса Кинопоиск

Преимущества Кинопоиска:

- 1 Ёмкий, лаконичный интерфейс.
- 2 Рекомендации похожих произведений.
- 3 Интеграция стримингового сервиса “Яндекс Музыка”, позволяющего прослушать саундтрек фильма прямо на его странице.
- 4 Доступ ко всей информации о произведении для незарегистрированных пользователей.

Shikimori — онлайн-сервис, предоставляющий доступ к базе данных о произведениях японской культуры.

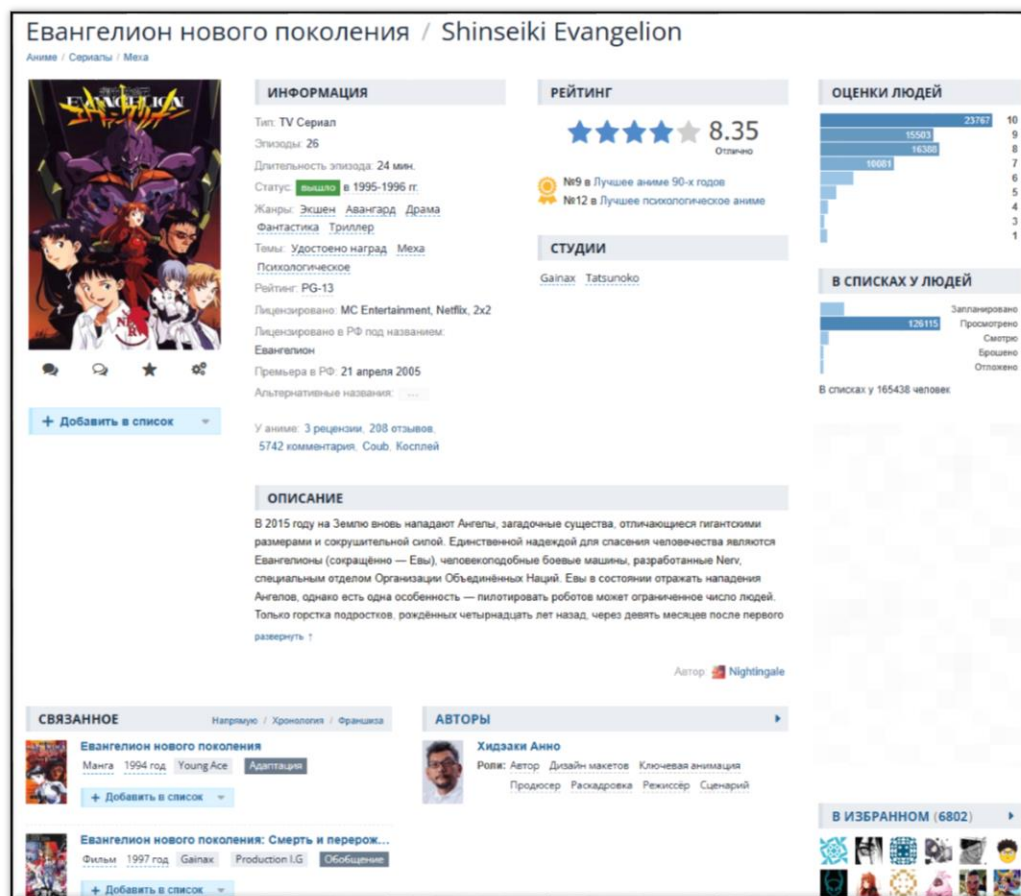


Рисунок 1.4 — пользовательский интерфейс сервиса Shikimori

Достоинства **Shikimori**:

- 1 Удобная навигация по веб-сайту.
- 2 Значительные возможности социального взаимодействия: возможность просмотра списков пользователей, наличие чата между пользователями ресурса.

Недостатки **Shikimori**:

- 1 Устаревший интерфейс.

2 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ

2.1 Обоснование необходимости разработки

Поскольку сфера кинематографа является одной из самых популярных в мире, веб-приложения, представляющие информацию об этой сфере развлечений, имеют большой спрос. После анализа существующих аналогов, изучения из достоинств и недостатков, выявлена ниша для сервиса, который позволит в простой и удобной для восприятия формы организовать просмотренные фильмы в список и выставить им оценки.

В современном цифровом пространстве кино контент потребляется в беспрецедентных масштабах: по данным Statista, мировой рынок онлайн-кинотеатров к 2027 году достигнет \$167 миллиардов. Однако избыток предложения создает проблему выбора для пользователей. Веб-приложение для оценки и рекомендации фильмов решает эту задачу, предоставляя персонализированный опыт на основе коллективного интеллекта и алгоритмов анализа данных.

Пользователи сталкиваются с информационной перегрузкой: ежегодно выпускается свыше 10 000 фильмов. Существующие платформы (IMDb, Кинопоиск) предлагают базовые рейтинги, но не всегда учитывают персональные предпочтения. Приложение с адаптивными рекомендациями (на основе жанров, режиссеров, настроения) закрывает этот пробел. Для бизнеса это инструмент аудиторного анализа: киностудии и стриминги могут использовать агрегированные данные для прогнозирования трендов.

Веб-приложение обеспечивает кроссплатформенную доступность без установки дополнительного ПО. Мгновенные обновления контента и алгоритмов возможны благодаря централизованному управлению серверной частью. Интеграция с социальными платформами (совместные просмотры, обзоры) усиливает вовлеченность.

Веб-решение требует меньших ресурсов для поддержки, чем мобильные аналоги. Монетизация возможна через:

- 1 Партнерские программы со стримингами.
- 2 Премиум-подписку.
- 3 Анонимизированные данные для маркетинговых исследований.

Разработка веб-приложения для оценки фильмов отвечает трем критическим требованиям: востребованность на рынке (персонализация контента), технологическая осуществимость (зрелость стеков веб-разработки) и образовательная ценность (комплексное применение навыков). Проект служит основой для будущего масштабирования: интеграции с IoT (умные телевизоры), голосовыми ассистентами или AR-витринами кинотеатров.

Помимо этого, разработка проекта позволит укрепить знания и практические навыки в разработке веб-приложений на платформе ASP.Net Core, а также улучшить знания в области проектирования проектов с использованием ООП.

2.2 Технологии программирования, используемые для решения поставленных задач

Для разработки backend веб-сайта используются следующие технологии программирования:

1 ASP.NET Core представляет собой современную, кроссплатформенную платформу для создания высокопроизводительных веб-приложений, разработанную компанией Microsoft. Она является эволюционным развитием ASP.NET, сохраняя лучшие черты предшественника, но предлагая принципиально новые архитектурные решения. Платформа активно используется для разработки API, веб-сайтов, микросервисов и облачных приложений, сочетая гибкость, масштабируемость и безопасность.

Одним из фундаментальных преимуществ ASP.NET Core является его кроссплатформенность. Приложения работают на Windows, Linux и macOS, что расширяет возможности развертывания и снижает зависимость от инфраструктуры. Платформа имеет открытый исходный код, доступный на GitHub, что способствует активному участию сообщества в ее развитии. Высокая производительность — еще один критически важный аспект; ASP.NET Core демонстрирует лучшие в своем классе показатели скорости обработки запросов и эффективности использования ресурсов, что подтверждается тестами TechEmpower.

Сердцем ASP.NET Core является модульная система Middleware, компоненты которой образуют конвейер обработки HTTP-запросов. Это позволяет разработчикам гибко настраивать поведение приложения, добавляя только необходимые функциональные слои (например, аутентификацию, кэширование или логирование). Платформа глубоко интегрирована с внедрением зависимостей (Dependency Injection), что упрощает управление объектами, повышает тестируемость кода и соблюдение принципов SOLID. Конфигурация приложения унифицирована и поддерживает множество источников: JSON-файлы, переменные окружения, базы данных.

В отличие от классического ASP.NET, новая платформа не зависит от System.Web.dll, что существенно снижает накладные расходы и ускоряет запуск. Модульность реализована через пакеты NuGet, позволяя включать в проект лишь требуемые функции, а не всю платформу целиком. Это сокращает размер приложений и поверхность атаки для угроз. Кроме того, ASP.NET Core поддерживает современные подходы, такие как контейнеризация (Docker) и облачные сценарии (Azure, AWS).

Для разработки используются языки C# или F#, фреймворк Entity Framework Core для работы с данными, а также Razor Pages или MVC для построения пользовательских интерфейсов. Платформа интегрируется с популярными инструментами: Visual Studio, Visual Studio Code, CLI. Благодаря встроенной поддержке асинхронного программирования, приложения эффективно обрабатывают параллельные запросы без блокировки потоков.

ASP.NET Core сочетает мощь корпоративных решений Microsoft с гибкостью open-source. Его архитектура, ориентированная на производительность и модульность, делает платформу идеальным выбором для проектов любого масштаба — от стартапов до высоконагруженных систем. С постоянным обновлением экосистемы (включая .NET 5+) и активным сообществом, ASP.NET Core продолжает задавать стандарты в веб-разработке [2].

2 ASP.NET Core MVC — это высокоструктурированный фреймворк в составе платформы ASP.NET Core, реализующий паттерн "Модель-Представление-Контроллер" (MVC). Он предназначен для создания сложных, масштабируемых и легко тестируемых веб-приложений с четким разделением ответственности между компонентами. В отличие от монолитных решений, MVC обеспечивает гибкость и контроль на каждом уровне приложения.

Фреймворк строго разделяет логику приложения на три взаимосвязанных компонента. Модели отвечают за бизнес-логику, данные и правила валидации, абстрагируя работу с источниками данных через ORM-инструменты вроде Entity Framework Core. Представления формируют пользовательский интерфейс с использованием HTML и Razor-синтаксиса, получая данные от контроллеров и минимизируя собственную логику. Контроллеры обрабатывают HTTP-запросы, координируют взаимодействие с моделями и определяют представления для рендеринга, выступая центральным посредником между пользователем и системой.

ASP.NET Core MVC наследует преимущества базовой платформы: кроссплатформенность, открытый исходный код и высокую производительность. Его уникальность проявляется в гибкой маршрутизации на основе атрибутов, позволяющей декларативно настраивать URL-шаблоны для действий контроллера. Фильтры (авторизации, обработки исключений, кэширования) предоставляют мощный механизм для сквозной функциональности. Привязка моделей автоматически преобразует данные запросов (формы, JSON, строки запроса) в объекты C#, сокращая рутинный код.

Архитектура MVC естественным образом способствует модульному тестированию. Контроллеры, лишённые прямой зависимости от UI или инфраструктуры, легко тестируются с помощью фреймворков вроде xUnit или NUnit через инъекцию зависимостей. Поддержка Razor-страниц предлагает альтернативу для сценариев с минимальной логикой контроллера. Валидация на стороне сервера и клиента (через jQuery или валидаторы на основе атрибутов) обеспечивает надёжность данных.

ASP.NET Core MVC остается эталоном для проектов, требующих предсказуемой структуры и высокой степени контроля. Сочетая проверенный паттерн MVC с инновациями .NET Core (производительность, кроссплатформенность), он предлагает баланс между строгостью архитектуры и адаптивностью к современным вызовам веб-разработки [3] [4].

3 C# (произносится как "си шарп") — это высокоуровневый, строго типизированный язык, разработанный Microsoft в рамках платформы .NET.

Созданный Андерсом Хейлсбергом в 2000 году, язык сочетает мощь C++ с простотой Java и активно развивается, оставаясь одним из наиболее востребованных инструментов для корпоративной разработки, игр (Unity), мобильных и веб-приложений.

C# проектировался как объектно-ориентированный язык, но с поддержкой функциональных, асинхронных и компонентно-ориентированных парадигм. Его ключевые принципы включают безопасность типов, производительность и читаемость кода. Язык следует правилу "разумных умолчаний": например, переменные инициализируются нулевыми значениями, а сборщик мусора автоматически управляет памятью, снижая риски утечек.

C# известен элегантным синтаксисом. Свойства с аксессорами get/set заменяют громоздкие методы доступа к полям. Делегаты и события реализуют шаблон наблюдателя для обработки действий. Лямбда-выражения позволяют писать лаконичный функциональный код. Индексаторы дают возможность обращаться к объектам как к массивам. Поддержка обобщений (generics) обеспечивает типобезопасные коллекции и алгоритмы.

Статическая типизация C# выявляет ошибки на этапе компиляции. Управляемый код выполняется в песочнице CLR (Common Language Runtime), что предотвращает прямой доступ к памяти. JIT-компиляция оптимизирует производительность, а AOT (в .NET Native) позволяет генерировать нативный код для сценариев с критическими требованиями к скорости.

C# доминирует в корпоративном секторе (банки, ERP), геймдеве (Unity), облачных сервисах (Azure). Его используют в IoT (Internet of Things) благодаря .NET MicroFramework. Совместимость с Docker и Kubernetes упрощает развертывание микросервисов.

C# — это динамично развивающийся язык, который адаптируется к современным вызовам: от облачных вычислений до искусственного интеллекта. Его баланс между простотой, строгостью и мощностью делает его идеальным выбором для сложных проектов. При поддержке Microsoft и открытого сообщества, C# продолжает задавать стандарты в индустрии.

4 Entity Framework Core — это современное средство сопоставления отношений объектов, которое позволяет создавать чистый, переносимый и высокоуровневый уровень доступа к данным с помощью .NET (C#) в различных базах данных, включая База данных SQL (локальной среде и Azure), SQLite, MySQL, PostgreSQL и Azure Cosmos DB. Он поддерживает запросы LINQ, отслеживание изменений, обновления и миграции схемы.

EF Core реализует подход Domain-Driven Design (DDD), отображая таблицы базы данных в классы C# (сущности), а связи между ними — в навигационные свойства. Ключевая цель — минимизировать "разрыв импеданса" между реляционной моделью данных и объектно-ориентированной логикой приложения. Инструмент поддерживает декларативную работу с данными через LINQ-запросы, которые транслируются в SQL, обеспечивая типобезопасность и предотвращая инъекции.

В основе EF Core лежит DbContext — центральный класс, управляющий подключением к БД, отслеживанием изменений сущностей и выполнением транзакций. Механизм Change Tracking автоматически определяет модификации объектов (добавление, обновление, удаление) и генерирует соответствующие SQL-команды. Подходы к проектированию включают Code-First (создание БД из классов C#) и Database-First (генерация кода из существующей схемы).

EF Core отличается гибкостью конфигурации: Fluent API и атрибуты данных позволяют тонко настраивать маппинг (ключи, индексы, связи 1:1, 1:N, N:N). Миграции — мощный инструмент для управления эволюцией схемы БД через версионные скрипты C#, интегрируемые в CI/CD. Поддержка поставщиков данных охватывает SQL Server, PostgreSQL, MySQL, SQLite, Oracle и даже NoSQL (Cosmos DB).

EF Core глубоко интегрирован с ASP.NET Core через Dependency Injection, что упрощает внедрение контекста в контроллеры. Шаблоны Repository Pattern и Unit of Work естественным образом реализуются на его основе. Инструменты Scaffolding в Visual Studio или CLI автоматически генерируют модели из БД.

EF Core применяется в корпоративных приложениях для построения многоуровневых архитектур с абстракцией доступа к данным

Entity Framework Core — это эволюция ORM для современной экосистемы .NET. Он сочетает простоту разработки с мощными возможностями настройки, оставаясь ключевым инструментом для эффективной работы с данными. При поддержке сообщества и постоянных обновлениях (включая улучшения в .NET 7-8), EF Core продолжает задавать стандарты в области объектно-реляционного отображения [5].

2.3 Связь архитектуры ООП с разрабатываемым проектом

Язык C#, как и его фреймворки, реализует парадигму объектно-ориентированного программирования. ООП — это парадигма программирования, сосредоточенная в первую очередь на инкапсуляции данных и методов работы с ними. Основные принципы ООП:

1 Абстракция: Моделирование требуемых атрибутов и взаимодействий сущностей в виде классов для определения абстрактного представления системы.

2 Инкапсуляция: Скрытие внутреннего состояния и функций объекта и предоставление доступа только через открытый набор функций.

3 Наследование: Возможность создания новых абстракций на основе существующих.

4 Полиморфизм: Возможность реализации наследуемых свойств или методов отличающимися способами в рамках множества абстракций [6].

Это хорошо согласуется с основными концепциями архитектурного шаблона MVC (Модель-Представление-Контроллер), используемого в

ASP.NET Core MVC. Данный шаблон реализует принцип инкапсуляции на уровне всего приложения — Хранение данных, обработка запросов и генерация ответов происходит в структурно разных частях приложения. В приложении ASP.NET Core MVC имеются:

1 Модели: модели в шаблоне MVC, как правило, представлены в виде классов C#, которые являются основой ООП. Данные классы при помощи ORM системы Entity Framework превращаются в таблицы базы данных.

2 Представления: представления в шаблоне MVC отвечают за отрисовку пользовательского интерфейса. Хотя представления напрямую не взаимодействуют с ООП, они взаимодействуют с объектами модели для отображения соответствующих данных.

3 Контроллеры: контроллеры в шаблоне MVC выступают посредником между моделями и представлениями. Контроллеры реализуются как классы C#, инкапсулирующие объект сеанса с базой данных и методы обработки поступающих запросов от пользователя в виде http-запросов.

Типичные проблемы ООП включают нарушение инкапсуляции (публичные поля вместо свойств), жёсткие зависимости между модулями, наследование ради повторного использования (вместо композиции), анемичные модели (классы-структуры без поведения). Антипаттерн God Object (класс, делающий всё) нарушает SRP и усложняет тестирование.

С развитием функционального программирования ООП адаптирует гибридные подходы: иммутабельность (record-типы в C#), реактивные модели (Rx.NET). В облачных средах актуальны контейнеризация (Docker) и serverless. Принципы ООА лежат в основе фреймворков: Spring (Java), ASP.NET Core, Ruby on Rails.

Помимо прочего, внедрение принципов ООП в приложения, написанные на ASP.Net Core помогают выделять повторяющиеся части кода и абстрагировать сущности для последующей их модификации без изменения бизнес-логики.

Чистая Архитектура (Clean Architecture) — это эволюционный подход к проектированию программных систем, предложенный Робертом Мартином (Uncle Bob). Её цель — создание устойчивых, тестируемых и независимых от внешних зависимостей приложений через строгое разделение ответственности и контроль направления зависимостей. Ключевая идея заключается в том, что бизнес-логика должна быть изолирована от инфраструктурных деталей (баз данных, фреймворков, UI), что позволяет системе эволюционировать без переписывания ядра.

Архитектура строится на правиле зависимостей (Dependency Rule): зависимости направлены внутрь, к центру системы. Внешние слои (инфраструктура, UI) зависят от внутренних (бизнес-правила), но не наоборот. Это достигается через инверсию зависимостей (DIP), где интерфейсы, определяемые ядром, реализуются во внешних слоях. Тестируемость становится тривиальной: бизнес-логика проверяется юнит-тестами без

поднятия БД или веб-сервера. Замена технологий (например, миграция с SQL на NoSQL) не затрагивает ядро системы.

В экосистеме .NET Clean Architecture воплощается через:

1 Domain-Driven Design (DDD) для моделирования сложной бизнес-логики.

2 CQRS (Command Query Responsibility Segregation) для разделения операций записи и чтения.

3 MediatR как реализацию паттерна "Посредник" для обработки команд и запросов.

4 Dependency Injection для связывания абстракций с реализациями. Например, интерфейс IUserRepository из домена реализуется классом UserRepository в инфраструктуре, который работает с EF Core.

Главное преимущество — долгосрочная поддерживаемость. Изменения в UI или БД не ломают тесты домена. Система устойчива к устареванию технологий. Однако архитектура требует дисциплины: строгое следование слоям увеличивает объем "боILERплейта". Для простых CRUD-приложений она может быть избыточной. Оптимальна для проектов со сложной предметной областью (финансовые системы, медицинские ПО).

Чистая Архитектура и ООП — взаимодополняющие парадигмы. Первая обеспечивает независимость от технологий через инверсию зависимостей, вторая предоставляет инструменты для моделирования сложных систем. Их комбинация позволяет создавать решения, которые легко тестировать, поддерживать и масштабировать в условиях меняющихся требований. Мастерство разработчика заключается в выборе баланса: когда следовать догмам, а когда — прагматично упрощать дизайн.

3 ФУНКЦИОНАЛЬНЫЕ ВОЗМОЖНОСТИ

3.1 Описание структуры базы данных приложения

Рассмотрим структуру классов, которые интерпретируются технологией ADO.NET Entity Framework как таблицы базы данных PostgreSQL.

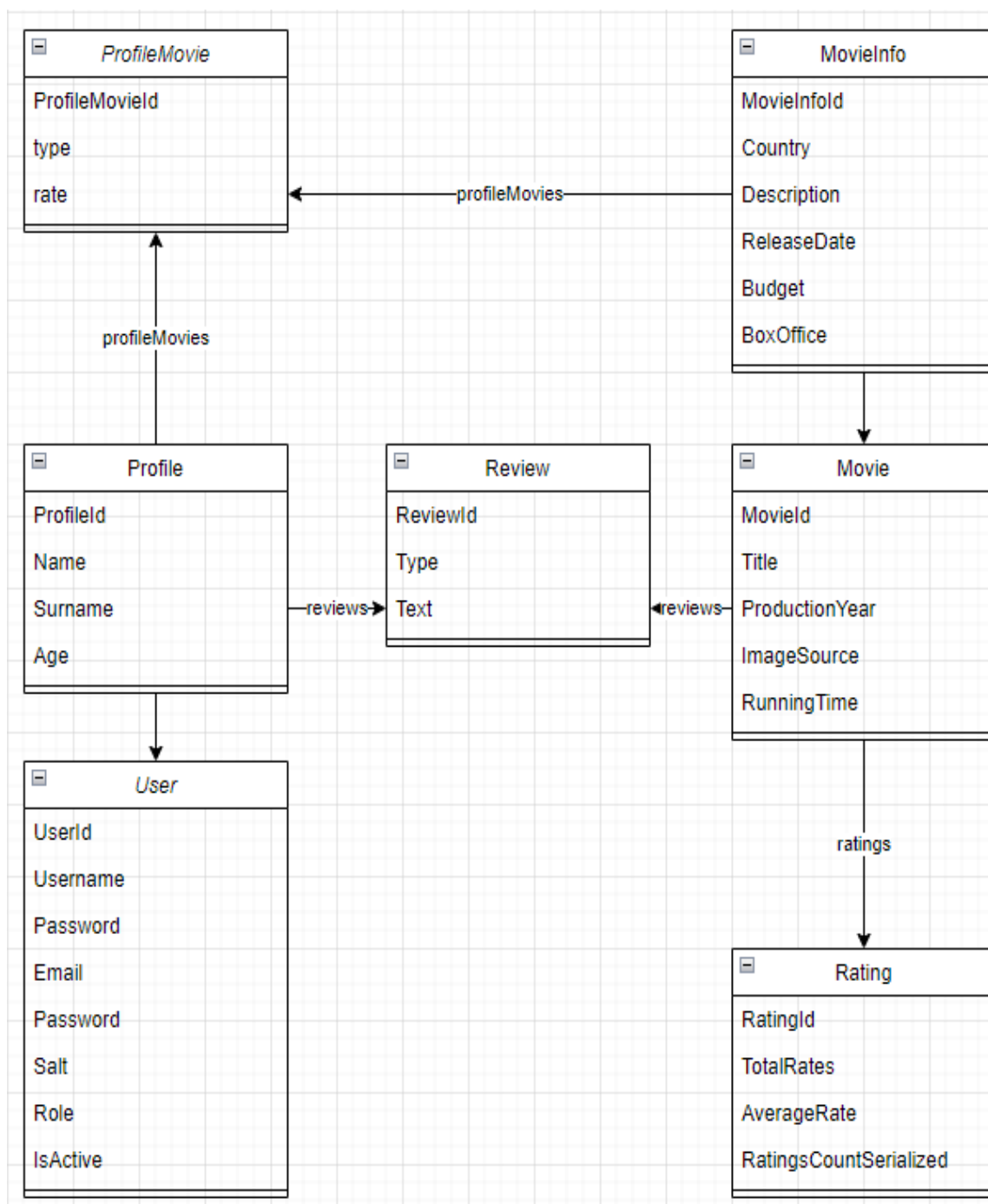


Рисунок 3.1 — Структура классов базы данных приложения

Класса `ApplicationContext` представляет собой контекст базы данных и отвечающий за сопоставление классов и таблиц базы данных, а также позволяющий обращаться к данным при помощи LINQ-запросов и сохранять внесённые изменения единомоментно.

```
public class ApplicationContext: DbContext
{
    public DbSet<User> Users { get; set; }
    public DbSet<Profile> Profiles { get; set; }
    public DbSet<Movie> Movies { get; set; }
    public DbSet<MovieInfo> MoviesInfo { get; set; }
    public DbSet<Rating> Ratings { get; set; }
    public DbSet<ProfileMovie> ProfileMovies { get; set; }
    public DbSet<Review> Reviews { get; set; }

    public ApplicationContext(
DbContextOptions<ApplicationContext> options) : base(options) { }
}
```

Листинг 3.1 – Листинг класса `ApplicationContext`

Поля типа `DbSet<>` представляют таблицы базы данных.

Рассмотрим подробнее классы базы данных. Класс `User` содержит в себе служебную информацию о пользователе – его никнейм и почту, необходимые для аутентификации, захешированный пароль и соль для хеширования, роль пользователя и флаг активности. Подробная информация о пользователе содержится в классе `Profile`, а именно: имя и фамилия пользователя, его возраст. Также класс `Profile` содержит ссылки на зависимые таблицы `Review` и `ProfileMovie`. Отношение между таблицами `Profile` и `Client` – один-к-одному. Класс `Movie` предоставляет базовую информацию о фильме: его название, год выпуска, ссылку на изображение и длительность. Дочерняя таблица `MovieInfo` содержит дополнительную информацию, которая отображается на странице фильма – описание, бюджет, страна производства. Связь между таблицами `Movie` и `MovieInfo` – один-к-одному. Класс `Rating` служит для отображения информации об оценках фильма пользователями. В нём хранятся суммарное количество оценок всех типов, средняя оценка (хранится в формате `Decimal` для большей точности значения) и сериализуемый список оценок, разбитый по значениям оценок.

```
public class Rating
{
```



```

    public int RatingId { get; set; }
    public Int64 TotalRates { get; set; }
    public Decimal AverageRate { get; set; }

    public string? RatingsCountSerialized { get; set; }

    [NotMapped]
    public Int64[] RatingsCount
    {
        get =>
            Array.ConvertAll(RatingsCountSerialized.Split(','), Int64.Parse);
        set => RatingsCountSerialized = string.Join(",", value);
    }

    public int MovieId { get; set; }
    public Movie Movie { get; set; } = null!;
}

```

Листинг 3.2 – Листинг класса Rating

Список оценок хранится в виде строки, разбитой разделителем со значением символа “,” (запятая). Для доступа к данным в этой строке используется не отображаемое в поле таблицы свойство RatingCount. Геттер этого свойства десериализует строку в массив значений типа Int64 (предполагается, что количество оценок фильма может превышать максимально возможное значение для четырёхбайтного целого числа), а его сеттер сериализует массив в строку заданного формата. Данный способ позволяет хранить массивы в удобном строковом представлении. Класс Rating связан с классом Movie отношением один-к-одному. Поскольку класс Rating зависит от класса Movie, ссылка на объект класса Movie внутри объекта класса Rating обязана быть ненулевой. Это достигается при помощи указания значения null!, введенного в C# 8.0.

Класс ProfileMovie представляет собой промежуточную таблицу в отношении многие-ко-многим таблиц классов Profile и Movie, служащую для хранения оценки фильма пользователем, а также для добавления пользователем фильма в один из доступных списков. Данный класс имеет следующие поля: тип списка, оценка и ссылки на две родительские таблицы.

3.2 Описание функций программного обеспечения приложения с учетом выбранной темы курсового проекта

Рассмотрим файл AccountController.cs (Приложение А), представляющий контроллер для обработки запросов и генерации веб-страниц на основе шаблонов представлений. При помощи механизма инъекций языка программирования С# в конструктор класса AccountController передаётся объект класса ApplicationContext, созданный в управляющем файле Program.cs. Данный механизм позволяет вынести конкретную реализацию сервисов из классов, непосредственно отвечающих за работу приложения, а также предоставляет удобный способ по управлению сервисами приложения.

Для регистрации пользователя применяется асинхронный метод-обработчик Registration(UserViewModel userViewModel). Он принимает объект класса UserViewModel, представляющий форму на веб-странице. Данный метод проверяет валидность формы на стороне сервера, а также то, существует ли пользователем с таким никнеймом и электронной почтой. В случае успешной проверки запускается процедура регистрации нового пользователя. Создаются объекты классов User и Profile. Для обеспечения сохранности конфиденциальных данных пользователей применяется хеширование паролей с уникальной солью. За это отвечают методы GenerateSalt (создаёт случайную соль при помощи класса RandomNumberGenerator фреймворка ASP.Net Core) и SaltHashPassword (хеширование методами класса SHA512). Стоит отметить, что хеширование паролей уникальной солью делает невозможным создание единой так называемой “радужной таблицы” для всей базы данных, поскольку к каждому паролю добавляется особая добавка в виде случайной строки.

Метод Login(LoginViewModel loginViewModel) принимает форму класса LoginViewModel, проверяет как валидность введённых данных, так и наличие в базе данных пользователя с введённым никнеймом. В случае успешной проверки осуществляется установка кук следующим способом. Покажем на примере метода Login(), как происходит валидация пользовательского ввода:

```
@model MovieDB.Models.UserModels.LoginViewModel
<form method="post">
  <div asp-validation-summary="ModelOnly"></div>
  <p>
    <label asp-for="Username">Username</label> <br />
    <input type="text" asp-for="Username" />
    @Html.ValidationMessage("Username")
  </p>
  <p>
    <label asp-for="Password">Password</label> <br />
    <input type="password" asp-for="Password" />
    @Html.ValidationMessage("Password")
  </p>
```

```

    <p>
      <input type="submit" value="Send" class="login_button" />
    </p>
  </form>

```

Листинг 3.3 – Листинг представления Login.cshtml

Для уменьшения нагрузки на сервер, помимо серверной валидации, дополнительно применяется валидация на стороне клиента. Для этого используется комбинация тегов `asp-for="Password"`, указывающего имя модели и метода `@Html.ValidationMessage("Username")`, выводящего ошибки.

```

public class LoginViewModel
{
    [Display(Name="Логин")]
    [StringLength(64, MinimumLength = 3, ErrorMessage = "Длина никнейма
должна быть от 3 до 64 символов")]
    [Required(ErrorMessage = "Не указан никнейм")]
    public string? Username { get; set; }

    [Display(Name = "Пароль")]
    [Required(ErrorMessage = "Не указан пароль")]
    [PasswordPropertyText]
    [StringLength(64, MinimumLength = 8, ErrorMessage = "Длина пароля
должна быть от 8 до 64 символов")]
    public string? Password { get; set; }
}

```

Листинг 3.4 – Листинг модели LoginViewModel

Для задания параметров валидации применяются встроенные в ASP.Net Core атрибуты валидации, задающие условие проверки и текст ошибки при некорректном вводе.

```

var claims = new List<Claim>
{
    new Claim(ClaimTypes.Name, user.Username),
    new Claim(ClaimTypes.Hash, user.Password),
    new Claim(ClaimTypes.Role, user.Role)
};
ClaimsIdentity claimsIdentity = new(claims, "Cookies");

```

```
ClaimsPrincipal claimsPrincipal = new(claimsIdentity);  
await HttpContext.SignInAsync("Cookies", claimsPrincipal);
```

Листинг кода 3.5 – Листинг метода Login(LoginViewModel loginViewModel)

Сперва создаются утверждения, содержащие определённые данные о пользователе: его никнейм, хеш пароля и роль. Затем на основе утверждений создаётся идентичность и при помощи объекта HttpContext записывается в куки браузера.

Метод Profile() создаёт страницу, которая отображает данные пользователя. Поскольку данные содержатся сразу в двух классах – User и Profile, необходимо создать объект специального класса ProfileViewModel, упрощающего отображения данной информации в соответствующем представлении.

Рассмотрим файл MoviesController (Приложение А), представляющий контроллер для обработки запросов, связанных с кинофильмами. Метод Movies() обрабатывает запросы типа HttpGet и создаёт html-страницу со списком всех фильмов. Для получения информации происходит обращения к контексту базы данных при помощи LINQ-запроса.

Важно отметить, что Entity Framework использует так называемую “ленивую” загрузку – это значит, что данные из дочерних таблиц подгружаться не будут. Поскольку данный метод, помимо прочего, возвращает и данные о рейтинге фильма, которые хранятся в таблице Rating, зависимой от Movies, необходимо применить LINQ-метод Include, явно указывающий, что данные из дочерней таблицы необходимо подгрузить.

```
@model      IEnumerable<MovieDB.Models.MovieModels.MovieCardViewModel>  
<style>  
    .movies_container {  
        display: grid;  
        grid-template-columns: repeat(auto-fill, minmax(300px, max-content));  
        justify-content: center;  
        gap: 20px;  
    }  
    .movie_card {  
        display: flex;  
        flex-direction: column;  
        position: relative;  
        width: 300px;  
        height: 450px;
```

```

    }
</style>


### Листинг кода 3.6 — Листинг представления Movies.cshtml



В данном представлении используется синтаксис Razor для генерации шаблонов веб-страниц. При помощи ключевого слова foreach происходит перебор и отображение всех элементов модели MovieCardViewModel, передаваемое методом-обработчиком контроллера. Карточка фильма представляет собой flex-контейнер заданного размера. Карточки организованы в сетку, автоматически генерируемую при помощи элемента grid. Данный способ позволяет отображать разное количество карточек заданного размера в зависимости от размера окна браузера.



Метод Movie(int? id) Принимает id фильма из базы данных и служит для отображения информации о нём. После успешной проверки, что фильм с таким id находится в базе данных, в соответствии с рекомендациями по проектированию приложений на фреймворке ASP.Net Core MVC создаётся объект специального класса-представления EditMovieViewModel, который хранит всю отображаемую информацию о фильме из двух классов: Movie и MovieInfo. Вместе с этим готовится информация для отображения графика о количестве и типах оценок, оставленных пользователями.



```

@model MovieDB.Models.MovieModels.EditMovieViewModel
.info_chart {
 width: 80%;
 height: auto;
 display: flex;

```



23


```

```

        flex-direction: column;
        justify-content: flex-start;
    }
    .info_chart > * {
        display: flex;
        align-content: center;
        justify-content: flex-end;
        height: 25px;
        background: DarkSlateBlue;
        margin: 4px;
        padding-right: 5px;
        color: white;
        font-size: 14px;
        animation: growWidth 1s ease-in forwards;
    }
    <div class="info_chart">
        @foreach (var widthAmountPair in (List<(int width, Int64
amount)>)ViewBag.WidthAmountPair)
        {
            @if (widthAmountPair.width > 15)
            {
                <div style="max-width: @widthAmountPair.width%; animation:
growWidth 2s ease-in-out forwards;">@widthAmountPair.amount</div>
            }
            else
            {
                <div style="max-width: @widthAmountPair.width%; animation:
growWidth 2s ease-in-out forwards;"></div>
            }
        }
    }
</div>

```

Листинг кода 3.7 – Листинг представления Movie.cshtml

График оценок реализован в виде контейнера flex, элементы которого представляют закрашенные элементы div с числом оценок данного типа, выровненными по правому краю. Ширина данных столбцов определяется в методе Movie(int? id) следующим образом: за максимально допустимую ширину принимается тип оценки с максимальным количеством голосов. Затем для каждой оценки рассчитывается процентное отношение к текущему максимуму. Результаты вычислений записываются в массив типа int. Затем

создаётся массив кортежей, хранящий пары “ширина – количество голосов”. Данный загружается во ViewBag (динамический массив для временного хранения данных и передачи их в представления).

Метод `AddToProfileList(int id, string WatchStatus, int Rate)` отвечает за добавление пользователем фильма в свой список и публикацией оценки. Метод принимает `id` фильма, а также данные формы, передающей строку с типом списка, в который нужно добавить фильм, и оценку фильма. Сперва проверяется, авторизован ли пользователь (при помощи атрибута `Authorize` метода) и существует ли фильм с таким `id`. Затем контекст базы данных загружает всю нужную информацию. После этого, в зависимости от выбранного пользователем списка (либо опцией “убрать из отслеживаемого”) происходит изменение массива с оценками фильма и пересчёт средней оценки. Для оптимизации нагрузки оценки было принято решение пересчитывать результат следующим образом: $(\text{средняя оценка} * \text{количество оценок} + \text{новая оценка}) / (\text{количество оценок} + 1)$.

Затем все внесённые изменения сохраняются в контексте базы данных. Для увеличения отзывчивости приложения используется асинхронный вариант метода.

Метод `UsersMovieList()` отображает статистику оценок пользователя и список отмеченных им фильмов. Как и метод `Profile()`, требует авторизации. Метод вызывает контекст базы данных и получает из него все записи об оценках фильма класса `ProfileMovie`, в котором есть ссылка на текущего пользователя. Затем формируются список всех отмеченных фильмов и информация для корректного отображения графика, подобно соответствующему методу из метода `Movie()` Того же контроллера. Для упрощения дизайна метод исключает те типы оценок, которые не содержат ни одной записи от пользователя.

```
@model
IEnumerable<MovieDB.Models.MovieModels.ProfileMovieViewModel>
<div style="margin-top: 25px;">
    @foreach (var statusType in ViewBag.WatchStatuses)
    {
        @if (WatchStatusesPair[statusType])
        {
            <div class="watch_status_container">
                <h4
class="watch_status_type">@StatusTypeRus[statusType] </h4>
                <h4 class="watch_status_rate">Оценка </h4>
            </div>
            <table class="movies_table">
```

```

        @foreach (var movie in Model)
        {
            @if(movie.type == statusType)
            {
                <tr>
                <th>
                <a class="movie_link" asp-controller="Movies" asp-
action="Movie" asp-route-id="@movie.MovieId">
                    @movie.title
                </a>
                </th>
                <td>
                <label>@movie.rate</label>
                </td>
                </tr>
            }
        }
    </table>
}
}
</div>

```

Листинг 3.8 — Листинг представления UsersMovieList.cshtml

Информация о фильмах отображается в виде таблицы. Название каждого фильма является ссылкой, что позволяет переходить на страницы их сразу из списка пользователя. В дополнение к уже рассмотренному элементу `foreach`, перебирающему элементы коллекции, используется конструкция `if`. Благодаря ей в списках пользователя отображаются только те категории, в которые добавлены фильмы.

3.3 Описание функциональной схемы программы

Функциональная схема программы — это визуализированное описание взаимодействия компонентов системы, потоков данных и бизнес-процессов без привязки к технической реализации. Ее разработка является критическим этапом проектирования, особенно для комплексных решений (таких как веб-приложение для оценки фильмов), обеспечивая ясность, предотвращение ошибок и эффективную коммуникацию между участниками проекта.

Схема декомпозирует систему на логические модули (аутентификация, рекомендации, рейтинги), демонстрируя взаимосвязи между ними. Например,

она показывает, как событие «пользователь оценил фильм» инициирует обновление рейтинга, генерацию персональных рекомендаций и запись в историю просмотров. Это выявляет узкие места (например, нагрузку на БД при массовых оценках) до написания кода, минимизируя дорогостоящие переделки на поздних этапах. Для образовательных проектов схема становится «дорожной картой», упрощающей понимание распределения задач в команде.

Схема служит живой документацией для новых разработчиков, сокращая время вхождения в проект. При модификациях (добавление чата обсуждений фильмов) она показывает точки интеграции, сохраняя целостность системы. В курсовой работе это демонстрирует системное мышление и снижает объем текстового описания.

При открытии веб-приложения неавторизованному пользователю будут доступны следующие действия: просмотр списка фильмов, изучение информации и оценок авторизованных пользователей о каждом фильме, регистрация и вход в систему. Авторизованный пользователь сможет выйти из системы, поставить фильму оценку и добавить его в свой список, просмотр своих оценок и своих списков.

Техническое изложение вышеописанного функционала для пользователя представлено на use-case диаграмме на рисунке 3.2.

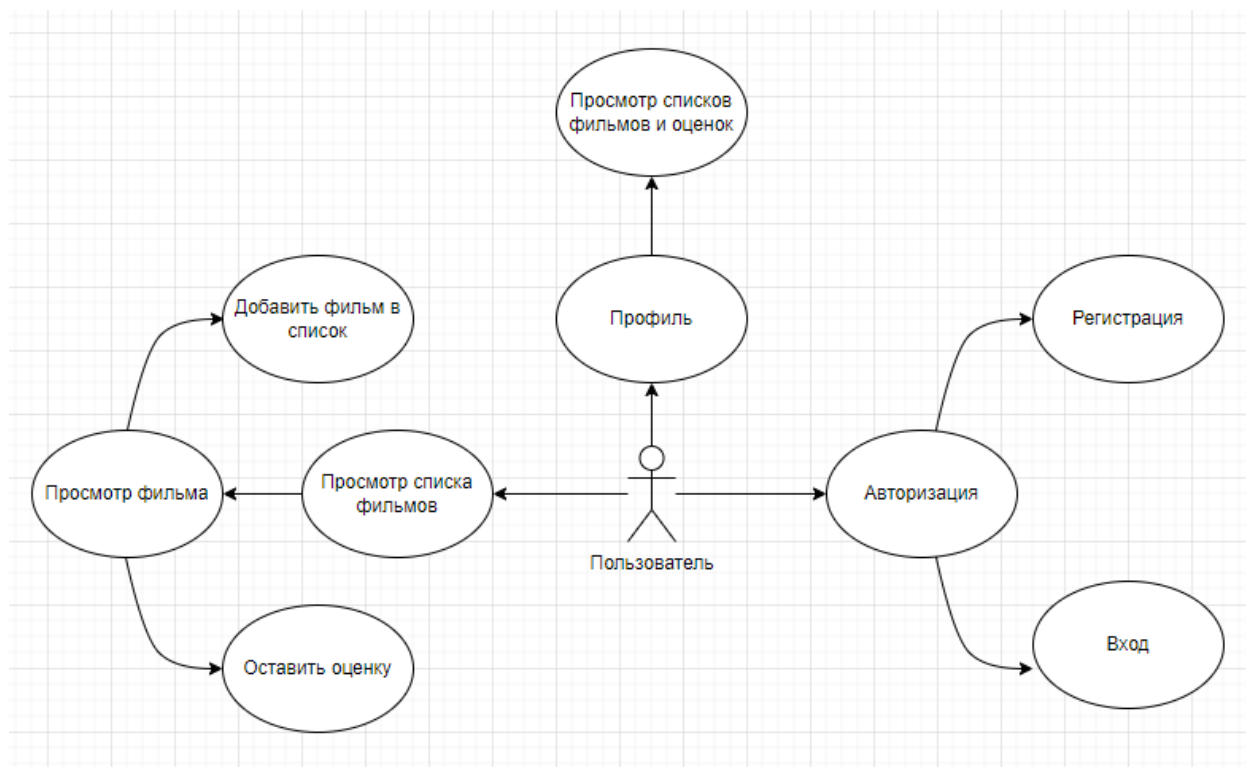


Рисунок 3.2 – Use-case диаграмма для пользователя

4 ОПИСАНИЕ ИСПОЛЬЗОВАНИЯ ПРИЛОЖЕНИЯ

Приложение MovieDB представляет собой уникальную платформу для оценивания фильмов, предоставления пользователю такой важной информации о фильме как:

- 1 Название.
- 2 Бюджет.
- 3 Кассовые сборы.
- 4 Страна производства.
- 5 Краткий сюжет.

При переходе во вкладку Movies, пользователь видит перед собой список фильмов из базы данных веб-приложения:

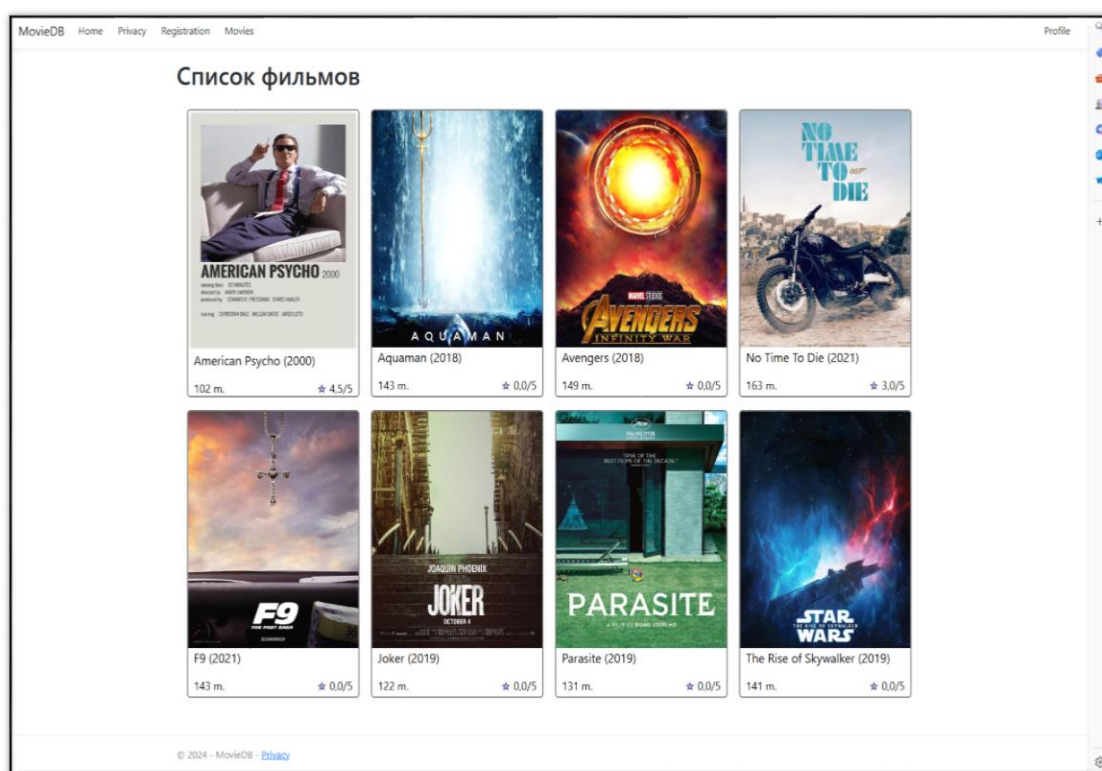


Рисунок 4.1 – Страница просмотра фильмов

В каждой карточке фильма содержится его название, год выпуска, хронометраж и средняя оценка всех пользователей. При переходе на страницу фильма пользователь видит информацию о нём, описание, а также статистику оценок:



Рисунок 4.2 – Страница фильма

При попытке неавторизованного пользователя оценить фильм или добавить его в список, веб-приложение перенаправит его на страницу входа. Аналогично, неавторизованный пользователь может самостоятельно посетить страницы входа и регистрации, выбрав соответствующие пункты в верхнем меню веб-приложения.

The screenshot shows a login form titled 'Войти' (Login). The form is enclosed in a rounded rectangle. It contains two input fields: 'Username' and 'Password'. Below these fields is a 'Send' button. The form is simple and clean, with a white background and black text.

Рисунок 4.3 – Форма входа в систему

При нарушении правил валидации или при вводе никнейма пользователя, которого нет в базе данных, приложение вернёт эту же страницу с ошибками валидации:

Войти

Username
 Пользователя с ником not_exist не существует

Password

Рисунок 4.4 – Форма входа в систему с ошибками валидации

Пользователь может зарегистрировать новый аккаунт, перейдя по соответствующей ссылке. Аналогично форме для входа, при некорректном вводе будет возвращена форма с дополнительной информацией о неуспешной валидации.

Регистрация

Username
 Не указан никнейм

Email
 Не указан электронный адрес

Password
 Пароль должен содержать буквы в обоих регистрах, цифры и специальные символы

PasswordConfirm
 Пароли не совпадают

Name
*Опционально

Surname
*Опционально

Age
 Не указан возраст

Рисунок 4.5 – Форма регистрации с ошибками валидации

При успешных регистрации или входе в систему пользователь будет перенаправлен на страницу профиля:

The screenshot shows a user profile page. At the top, the title "Профиль test_user3" is displayed. Below it, the email "Почта: test_user3@mail.ru" and age "Возраст: 22" are shown. At the bottom, there are four links: "Ваш список фильмов", "Изменить данные", "Изменить пароль", and "Выйти".

Профиль test_user3

Почта: test_user3@mail.ru

Возраст: 22

[Ваш список фильмов](#) [Изменить данные](#) [Изменить пароль](#) [Выйти](#)

Рисунок 4.6 – Профиль пользователя

В профиле можно перейти на страницы изменения данных аккаунта и пароля. Рассмотрим возможность изменения личных данных.

The screenshot shows a form titled "Изменение профиля". It contains input fields for "Email" (test_user3@mail.ru), "Name" (Анатолий), "Surname" (Артамонов), and "Age" (32). There are also labels for "Email", "Name", "Surname", and "Age", and optional labels "*Опционально" for "Name" and "Surname". A "Send" button is at the bottom.

Изменение профиля

Email
test_user3@mail.ru

Name
*Опционально
Анатолий

Surname
*Опционально
Артамонов

Age
32

[Send](#)

Рисунок 4.7 – Форма изменения личных данных пользователя

После изменения данных, пользователь перенаправляется на страницу профиля с уже применёнными изменениями:

The screenshot shows a user profile interface. At the top, the title 'Профиль test_user3' is displayed in a large, bold, black font. Below the title, the user's email 'test_user3@mail.ru' is shown next to the label 'Почта:'. The user's name 'Анатолий' is shown next to the label 'Имя:'. The user's surname 'Артамонов' is shown next to the label 'Фамилия:'. The user's age '32' is shown next to the label 'Возраст:'. At the bottom of the profile card, there are four navigation links: 'Ваш список фильмов', 'Изменить данные', 'Изменить пароль', and 'Выйти'.

Профиль test_user3

Почта: test_user3@mail.ru

Имя:
Анатолий

Фамилия:
Артамонов

Возраст: 32

Ваш список фильмов Изменить данные Изменить пароль Выйти

Рисунок 4.8 – Применение изменений профиля пользователя

Аналогичным образом меняется пароль: при этом применяются те же параметры валидации, что и при регистрации.

The screenshot shows a password change form. The title 'Изменение пароля' is displayed in a large, bold, black font. Below the title, there are two input fields. The first input field is labeled 'Новый пароль' and has a red border. To its right, a red error message reads: 'Пароль должен содержать буквы в обоих регистрах, цифры и специальные символы'. The second input field is labeled 'Подтвердите пароль' and also has a red border. To its right, a red error message reads: 'Пароли не совпадают'. At the bottom of the form, there is a button labeled 'Изменить'.

Изменение пароля

Новый пароль
• Пароль должен содержать буквы в обоих регистрах, цифры и специальные символы

Подтвердите пароль
• Пароли не совпадают

Изменить

Рисунок 4.9 – Форма изменения пароля аккаунта

Авторизованный пользователь может оценить фильм или добавить его в один из списков.

Оценки зрителей

1 5

1 4

3

2

1

Оценить

Добавить в список Оценка

Просмотрено 5

Отправить

5

1

2

3

4

5

Рисунок 4.10 – Добавление оценки фильма

После пользователь может перейти на вкладку со списком фильмов, которые он добавил в категории и ознакомиться со статистикой своих оценок.

Список фильмов		Ваши Оценки
Просмотрено	Оценка	1 5
American Psycho	5	1 3
Брошено	Оценка	
No Time To Die	3	

Рисунок 4.11 – Список фильмов и статистика оценок

Пользователь, авторизованный как администратор, может добавлять информацию о фильмах или изменять её для уже существующих.

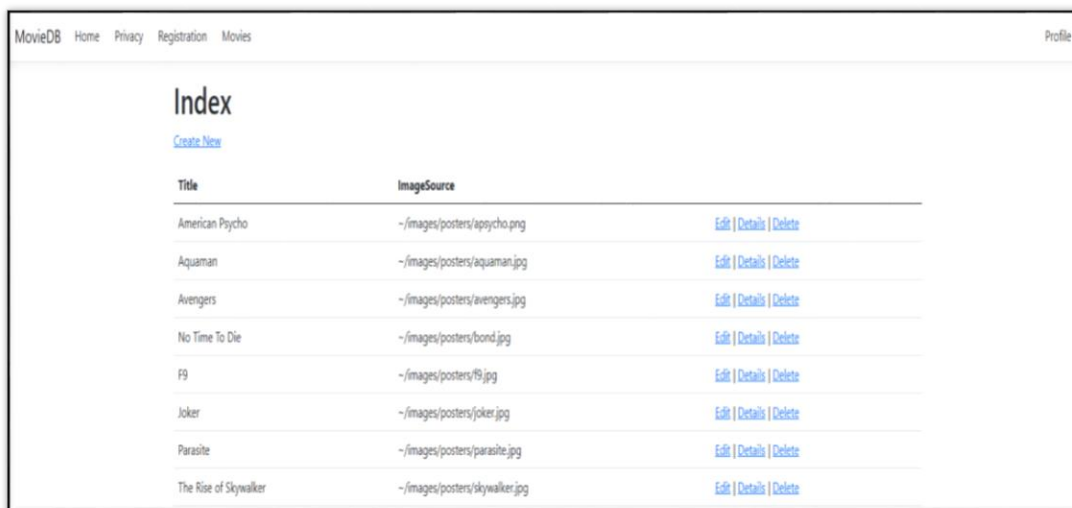


Рисунок 4.12 – Окно администратора

Также, администратор может удалять фильмы:

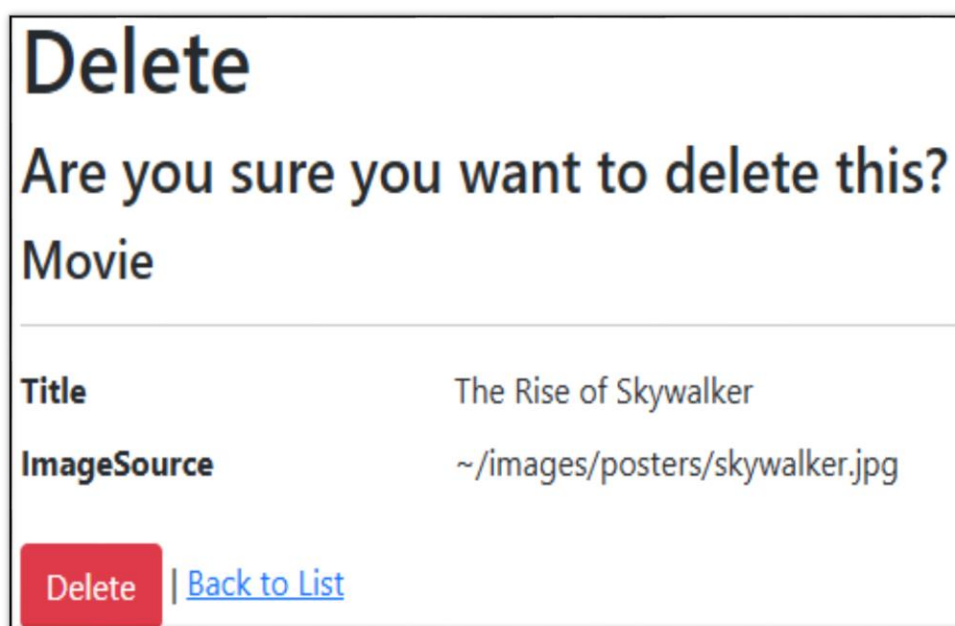


Рисунок 4.13 – Окно удаления записи о фильме администратором

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы были изучены представленные на рынке аналоги сервисов для получения информации в сфере киноиндустрии. Для реализации проекта была выбрана платформа ASP.Net Core MVC. Результатом работы является разработанное веб-приложение, предоставляющее информацию о фильмах, с возможностью их оценки и категоризации по различным спискам.

В проекте реализована собственная система авторизации и аутентификации с защитой паролей при помощи хеширования со случайной солью, работа с базой данных при помощи ORM инструмента Entity Framework.

При проектировании проекта активно использовались принципы ООП, SOLID и MVC. Для облегчения разработки Элементы проекта разбиты на следующие части: модели для связи с базой данных, представления как шаблоны для генерации html-страниц и контроллеры для обработки http-запросов. Для улучшения поддерживаемости проекта и реализации такого принципа ООП, как абстракция, использовался механизм внедрения зависимостей для реализации сеанса доступа к базе данных. Это позволит изменить провайдера базы данных без вмешательства в бизнес-логику приложения. Принятые в ходе проектирования решения на практике продемонстрировали своё преимущество в изолированности компонентов проекта при их изменении.

В процессе работы над курсовым проектом были улучшены знания в сфере программирования на языке C#, использования платформы ASP.Net Core, а также на практике оценены современные паттерны проектирования. Помимо этого, были получены навыки разработки пользовательских интерфейсов, вёрстки и стилизации страниц при помощи HTML и CSS.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

- [1] Доманов, А. Т. Стандарт предприятия / А. Т. Доманов, Н. И. Сорока. – Минск: БГУИР, 2024. – 178 с.
- [2] Документация ASP.NET Core [Электронный ресурс]. – Режим доступа: <https://docs.microsoft.com/en-us/aspnet/core/>
- [3] Документация ASP.NET Core MVC [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/en-us/aspnet/core/mvc/>
- [4] Документация Entity Framework [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/ef/>
- [5] Документация C# [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/en-us/dotnet/csharp/>
- [6] Документация CSS Mozilla [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org/ru/docs/Web/CSS>